

INTRODUCTION TO NUMPY

1. NumPy

- It stands for Numerical Python. It is a Python library used for working with arrays.

2. Why Use NumPy?

- In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide an array object that is up to 50x faster than traditional Python lists. The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy. Arrays are very frequently used in data science, where speed and resources are very important.
- Data Science: is a branch of computer science where we study how to store, use and analyze data for deriving information from it.

3. Why is NumPy Faster Than Lists?

- NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently. This behavior is called locality of reference in computer science.

4. Installation of NumPy

- To install we use command
`pip install numpy`
- Once NumPy is installed, import it in your applications by adding the import keyword

```
pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
import numpy as np
arr = np.array([1,2,3,4,5])
print(arr)
```

```
[1 2 3 4 5]
```

5. Checking NumPy Version

```
#The version string is stored under __version__ attribute.
import numpy as np
print(np.__version__)
```

```
2.0.2
```

NumPy Creating Arrays

1. Create a NumPy ndarray Object

- NumPy is used to work with arrays. The array object in NumPy is called ndarray. We can create a NumPy ndarray object by using the `array()` function.
- `type()`: This built-in Python function tells us the type of the object passed to it.

```
import numpy as np
arr = np.array([1,2,3,4,5])
print(arr)
print(type(arr))
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

- To create an ndarray, we can pass a list, tuple or any array-like object into the `array()` method, and it will be converted into an ndarray:

```
import numpy as np
arr = np.array((1,2,3,4,5))
print(arr)
```

```
[1 2 3 4 5]
```

2. Dimensions in Arrays

- A dimension in arrays is one level of array depth (nested arrays). Nested array are arrays that have arrays as their elements.

- **0-D Arrays** :- 0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

```
#Create a 0-D array with value 42
import numpy as np
arr = np.array(42)
print(arr)
```

```
42
```

- **1-D Arrays** :- An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.

```
#Create a 1-D array containing the values 1,2,3,4,5:
import numpy as np
arr = np.array([1,2,3,4,5])
print(arr)
```

```
[1 2 3 4 5]
```

- **2-D Arrays** :- An array that has 1-D arrays as its elements is called a 2-D array. These are often used to represent matrix or 2nd order tensors.

```
#Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:
import numpy as np
arr = np.array([[1,2,3],[4,5,6]])
print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

- **3-D arrays** :- An array that has 2-D arrays (matrices) as its elements is called 3-D array. These are often used to represent a 3rd order tensor.

```
#Create a 3-D array with two 2-D arrays,
#both containing two arrays with the values 1,2,3 and 4,5,6:
import numpy as np
arr = np.array([[[1,2,3],[4,5,6]], [[1,2,3],[4,5,6]]])
print(arr)
```

```
[[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

3. Check Number of Dimensions

- NumPy Arrays provides the `ndim` attribute that returns an integer that tells us how many dimensions the array have.

```
#Check how many dimensions the arrays have:
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

```
0
1
2
3
```

4. Higher Dimensional Arrays

- An array can have any number of dimensions. When the array is created, you can define the number of dimensions by using the `ndmin` argument.

```
#Create an array with 5 dimensions and verify that it has 5 dimensions:
import numpy as np
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('number of dimensions :', arr.ndim)
```

```
[[[[[1 2 3 4]]]]]  
number of dimensions : 5
```

- In this array the innermost dimension (5th dim) has 4 elements, the 4th dim has 1 element that is the vector, the 3rd dim has 1 element that is the matrix with the vector, the 2nd dim has 1 element that is 3D array and 1st dim has 1 element that is a 4D array.

NumPy Array Indexing

1. Access Array Elements

- Array indexing is the same as accessing an array element. You can access an array element by referring to its index number.
- The indexes in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1 etc.

2. Access 1-D Arrays

```
#Get the first element from the following array:  
import numpy as np  
arr = np.array([1, 2, 3, 4])  
print(arr[0])
```

1

3. Access 2-D Arrays

- To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.
- 2-D arrays are like table with rows and columns, where the dimension represents the row and the index represents the column.

```
#Access the element on the first row, second column:  
import numpy as np  
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])  
print('2nd element on 1st row: ', arr[0, 1])
```

2nd element on 1st row: 2

4. Access 3-D Arrays

- To access elements from 3-D arrays we can use comma separated integers representing the dimensions and the index of the element.
- The first number represents the first dimension, The second number represents the second dimension and The third number represents the third dimension.

```
import numpy as np  
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])  
print(arr[0, 1, 2])
```

6

5. Negative Indexing

- Using negative indexing to access an array from the end.

```
#Print the last element from the 2nd dim:  
import numpy as np  
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])  
print('Last element from 2nd dim: ', arr[1, -1])
```

Last element from 2nd dim: 10

NumPy Array Slicing

1. Slicing arrays

- Slicing in python means taking elements from one given index to another given index.
- [start:end:step]

If we don't pass start its considered 0

If we don't pass end its considered length of array in that dimension

If we don't pass step its considered 1

Note :- The result includes the start index, but excludes the end index.

```
#Slice elements from index 1 to index 5
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
```

```
[2 3 4 5]
```

```
#slice elements from index 4 to the end of the array:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[4:])
```

```
[5 6 7]
```

```
#Slice elements from the beginning to index 4 (not included):
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[:4])
```

```
[1 2 3 4]
```

2. Negative Slicing

- Using the minus operator to refer to an index from the end:

```
#Slice from the index 3 from the end to index 1 from the end:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[-3:-1])
```

```
[5 6]
```

- Slicing 2-D Arrays

```
#From the second element, slice elements from index 1 to index 4 (not included):
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[1, 1:4])
```

```
[7 8 9]
```

```
#From both elements, return index 2:
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 2])
```

```
[3 8]
```

```
#From both elements, slice index 1 to index 4 (not included),
#this will return a 2-D array:
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 1:4])
```

```
[[2 3 4]
 [7 8 9]]
```

NumPy Data Types

1. Data Types in Python

- strings - used to represent text data. e.g. "ABCD"
- integer - used to represent integer numbers. e.g. -1, -2, -3
- float - used to represent real numbers. e.g. 1.2, 42.42
- boolean - used to represent True or False.
- complex - used to represent complex numbers. e.g. 1.0 + 2.0j, 1.5 + 2.5j

2. Data Types in NumPy

- i - integer
- b - boolean
- u - unsigned integer
- f - float
- c - complex float
- m - timedelta

- M - datetime
- O - object
- S - string
- U - unicode string
- V - fixed chunk of memory for other type (void)

3. Checking the Data Type of an Array

- The NumPy array object has a property called dtype that returns the data type of the array

```
#Get the data type of an array object:
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr.dtype)
```

```
int64
```

```
import numpy as np
arr = np.array(['apple', 'banana', 'cherry'])
print(arr.dtype)
```

```
<U6
```

4. Creating Arrays With a Defined Data Type

- array() function take an optional argument: dtype that allows us to define the expected data type of the array elements

```
#Create an array with data type string:
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='S')
print(arr)
print(arr.dtype)
```

```
[b'1' b'2' b'3' b'4']
|S1
```

- For i, u, f, S and U we can define size as well.

```
#Create an array with data type 4 bytes integer:
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='i4')
print(arr)
print(arr.dtype)
```

```
[1 2 3 4]
int32
```

- If a type is given in which elements can't be casted then NumPy will raise a ValueError.

```
import numpy as np
#arr = np.array(['a', '2', '3'], dtype='i') #value error
```

5. Converting Data Type on Existing Arrays

- The best way to change the data type of an existing array, is to make a copy of the array with the astype() method.
- The astype() function creates a copy of the array, and allows you to specify the data type as a parameter.

```
import numpy as np
arr = np.array([1.1, 2.1, 3.1])
newarr = arr.astype('i')
print(newarr)
print(newarr.dtype)
```

```
[1 2 3]
int32
```

```
import numpy as np
arr = np.array([1, 0, 3])
newarr = arr.astype(bool)
print(newarr)
print(newarr.dtype)
```

```
[ True False  True]
bool
```

NumPy Array Copy vs View

1. The Difference Between Copy and View

- The copy is a new array, and the view is just a view of the original array.
- The copy owns the data and any changes made to the copy will not affect original array, and any changes made to the original array will not affect the copy.
- The view does not own the data and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

2. COPY

```
#Make a copy, change the original array, and display both arrays:
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42
print(arr)
print(x)
```

```
[42  2  3  4  5]
[1  2  3  4  5]
```

3. VIEW

```
#Make a view, change the original array, and display both arrays:
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42
print(arr)
print(x)
```

```
[42  2  3  4  5]
[42  2  3  4  5]
```

```
#Make a view, change the view, and display both arrays:
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
x[0] = 31
print(arr)
print(x)
```

```
[31  2  3  4  5]
[31  2  3  4  5]
```

4. Check if Array Owns its Data

- Every NumPy array has the attribute base that returns None if the array owns the data. Otherwise, the base attribute refers to the original object.

```
#Print the value of the base attribute to check if an array owns it's data or not:
import numpy as np
arr = np.array([1, 2, 3, 4, 5])

x = arr.copy()
y = arr.view()

print(x.base)
print(y.base)
```

```
None
[1  2  3  4  5]
```

NumPy Array Shape

1. Shape of an Array

- It is the number of elements in each dimension.

2. Get the Shape of an Array

- NumPy arrays have an attribute called shape that returns a tuple with each index having the number of corresponding elements.
- Integers at every index tells about the number of elements the corresponding dimension has.

```
#Print the shape of a 2-D array:
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape)
```

```
(2, 4)
```

```
import numpy as np
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('shape of array :', arr.shape)
```

```
[[[[[1 2 3 4]]]]]
shape of array : (1, 1, 1, 1, 4)
```

NumPy Array Reshaping

1. Reshaping arrays

- Reshaping means changing the shape of an array. The shape of an array is the number of elements in each dimension. By reshaping we can add or remove dimensions or change number of elements in each dimension.
- We can reshape into any shape as long as the elements required for reshaping are equal in both shapes.

2. Reshape From 1-D to 2-D

```
#The outermost dimension will have 4 arrays, each with 3 elements:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print(newarr)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

3. Reshape From 1-D to 3-D

```
#The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(2, 3, 2)
print(newarr)
```

```
[[[ 1  2]
   [ 3  4]
   [ 5  6]]

 [[ 7  8]
   [ 9 10]
   [11 12]]]
```

4. Returns Copy or View?

```
#Check if the returned array is a copy or a view:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
print(arr.reshape(2, 4).base) #returned array is a view
```

```
[1 2 3 4 5 6 7 8]
```

5. Unknown Dimension

- You are allowed to have one "unknown" dimension. Meaning that you do not have to specify an exact number for one of the dimensions in the reshape method.
- Pass -1 as the value, and NumPy will calculate this number for you.

```
#import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
newarr = arr.reshape(2, 2, -1)
print(newarr)
```

```
[[[1 2]
   [3 4]]

 [[5 6]
   [7 8]]]
```

6. Flattening the arrays

- Flattening array means converting a multidimensional array into a 1D array.

```
#Convert the array into a 1D array:
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
newarr = arr.reshape(-1)
print(newarr)
```

```
[1 2 3 4 5 6]
```

NumPy Array Iterating

1. Iterating Arrays

- Iterating means going through elements one by one.

2. Iterating 1-D Arrays

```
#Iterate on the elements of the following 1-D array:
import numpy as np
arr = np.array([1, 2, 3])
for x in arr:
    print(x)
```

```
1
2
3
```

3. Iterating 2-D Arrays

- If we iterate on a n-D array it will go through n-1th dimension one by one.

```
#Iterate on the elements of the following 2-D array:
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
    print(x)
```

```
[1 2 3]
[4 5 6]
```

- To return the actual values, the scalars, we have to iterate the arrays in each dimension.

```
#Iterate on each scalar element of the 2-D array:
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
    for y in x:
        print(y)
```

```
1
2
3
4
5
6
```

4. Iterating 3-D Arrays

- In a 3-D array it will go through all the 2-D arrays.

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
    print(x)
```

```
[[1 2 3]
 [4 5 6]]
[[ 7  8  9]
 [10 11 12]]
```

- To return the actual values, the scalars, we have to iterate the arrays in each dimension.

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
```



```
for x in arr:
    for y in x:
        for z in y:
            print(z)
```

```
1
2
3
4
5
6
7
8
9
10
11
12
```

5. Iterating Arrays Using nditer()

```
import numpy as np
arr = np.array([[1, 2], [3, 4]], [[5, 6], [7, 8]])
for x in np.nditer(arr):
    print(x)
```

```
1
2
3
4
5
6
7
8
```

6. Iterating Array With Different Data Types

```
import numpy as np
arr = np.array([1, 2, 3])
for x in np.nditer(arr, flags=['buffered'], op_dtypes=['S']):
    print(x)
```

```
np.bytes_(b'1')
np.bytes_(b'2')
np.bytes_(b'3')
```

7. Iterating Array With Different Step Size

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for x in np.nditer(arr[:, ::2]):
    print(x)
```

```
1
3
5
7
```

8. Enumerated Iteration Using ndenumerate()

- Enumeration means mentioning sequence number of somethings one by one.
- Sometimes we require corresponding index of the element while iterating, the ndenumerate() method can be used for those usecases.

```
import numpy as np
arr = np.array([1, 2, 3])
for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

```
(0,) 1
(1,) 2
(2,) 3
```

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

```
(0, 0) 1
(0, 1) 2
```

```
(0, 2) 3
(0, 3) 4
(1, 0) 5
(1, 1) 6
(1, 2) 7
(1, 3) 8
```

NumPy Joining Array

1. Joining NumPy Arrays

- Joining means putting contents of two or more arrays in a single array.
- In SQL we join tables based on a key, whereas in NumPy we join arrays by axes.
- We pass a sequence of arrays that we want to join to the concatenate() function, along with the axis. If axis is not explicitly passed, it is taken as 0.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.concatenate((arr1, arr2))
print(arr)
```

```
[1 2 3 4 5 6]
```

```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

arr = np.concatenate((arr1, arr2), axis=1)

print(arr)
```

```
[[1 2 5 6]
 [3 4 7 8]]
```

2. Joining Arrays Using Stack Functions

- Stacking is same as concatenation, the only difference is that stacking is done along a new axis.
- We can concatenate two 1-D arrays along the second axis which would result in putting them one over the other, ie. stacking.
- We pass a sequence of arrays that we want to join to the stack() method along with the axis. If axis is not explicitly passed it is taken as 0.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.stack((arr1, arr2), axis=1)
print(arr)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

3. Stacking Along Rows Using hstack()

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.hstack((arr1, arr2))
print(arr)
```

```
[1 2 3 4 5 6]
```

4. Stacking Along Columns Using vstack()

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.vstack((arr1, arr2))
print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

5. Stacking Along Height (depth) using dstack()

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.dstack((arr1, arr2))
print(arr)
```

```
[[[1 4]
  [2 5]
  [3 6]]]
```

NumPy Splitting Array

1. Splitting NumPy Arrays

- Splitting is reverse operation of Joining.
- We use `array_split()` for splitting arrays, we pass it the array we want to split and the number of splits.

```
#Split the array in 3 parts:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr) #Note: The return value is a list containing three arrays.
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

- If the array has less elements than required, it will adjust from the end accordingly.

```
#Split the array in 4 parts:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 4)
print(newarr)
```

```
[array([1, 2]), array([3, 4]), array([5]), array([6])]
```

2. Splitting 1-D Arrays

- The return value of the `array_split()` method is a list containing each of the split as an array. If you split an array into 3 arrays, you can access them from the result just like any array element:

```
#Access the splitted arrays:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr[0])
print(newarr[1])
print(newarr[2])
```

```
[1 2]
[3 4]
[5 6]
```

3. Splitting 2-D Arrays

```
#Split the 2-D array into three 2-D arrays.
import numpy as np
arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
newarr = np.array_split(arr, 3)
print(newarr)
```

```
[array([[1, 2],
        [3, 4]]), array([[5, 6],
        [7, 8]]), array([[9, 10],
        [11, 12]])]
```

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15], [16, 17, 18]])
newarr = np.array_split(arr, 3)
print(newarr)
```

```
[array([[1, 2, 3],
        [4, 5, 6]]), array([[7, 8, 9],
        [10, 11, 12]]), array([[13, 14, 15],
        [16, 17, 18]])]
```

4. `hsplit()` opposite of `hstack()`

```
#Use the hsplit() method to split the 2-D array into three 2-D arrays along columns.
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15], [16, 17, 18]])
newarr = np.hsplit(arr, 3)
print(newarr)
```

```
[array([[ 1],
       [ 4],
       [ 7],
       [10],
       [13],
       [16]]), array([[ 2],
       [ 5],
       [ 8],
       [11],
       [14],
       [17]]), array([[ 3],
       [ 6],
       [ 9],
       [12],
       [15],
       [18]])]
```

5. vsplit()

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15], [16, 17, 18]])
newarr = np.vsplit(arr, 3)
print(newarr)
```

```
[array([[1, 2, 3],
       [4, 5, 6]]), array([[ 7,  8,  9],
       [10, 11, 12]]), array([[13, 14, 15],
       [16, 17, 18]])]
```

6. dsplit() #work only on 3-d arrays

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15], [16, 17, 18]]])
newarr = np.dsplit(arr, 3)
print(newarr)
```

```
[array([[[ 1],
         [ 4],
         [ 7],
         [10],
         [13],
         [16]]]), array([[[ 2],
         [ 5],
         [ 8],
         [11],
         [14],
         [17]]]), array([[[ 3],
         [ 6],
         [ 9],
         [12],
         [15],
         [18]]])]
```

NumPy Searching Arrays

1. Searching Arrays

- You can search an array for a certain value, and return the indexes that get a match. To search an array, use the where() method.

```
#Find the indexes where the value is 4:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x) #return a tuple: (array([3, 5, 6]),) Which means that the value 4 is present at index 3, 5, and 6.
```

```
(array([3, 5, 6]),)
```

2. Search Sorted

- There is a method called searchsorted() which performs a binary search in the array, and returns the index where the specified value would be inserted to maintain the search order.

```
import numpy as np
arr = np.array([6, 7, 8, 9])
x = np.searchsorted(arr, 7)
print(x) # The number 7 should be inserted on index 1 to remain the sort order.
#The method starts the search from the left and returns the first index where the number 7 is no longer larger than the next
```

1

- By default the left most index is returned, but we can give side='right' to return the right most index instead.

```
import numpy as np
arr = np.array([5, 6, 7, 8, 9])
x = np.searchsorted(arr, 7, side='right')
print(x) #The number 7 should be inserted on index 2 to remain the sort order.
#The method starts the search from the right and returns the first index where the number 7 is no longer less than the next
```

3

- To search for more than one value, use an array with the specified values.

```
import numpy as np
arr = np.array([1, 3, 5, 7])
x = np.searchsorted(arr, [2, 4, 6])
print(x)
```

[1 2 3]

NumPy Sorting Arrays

1. Sorting Arrays

- Sorting means putting elements in an ordered sequence.
- The NumPy ndarray object has a function called sort(), that will sort a specified array.

```
#Sort the array:
import numpy as np
arr = np.array([3, 2, 0, 1])
print(np.sort(arr))
```

[0 1 2 3]

- Note: This method returns a copy of the array, leaving the original array unchanged.

```
#Sort the array alphabetically:
import numpy as np
arr = np.array(['banana', 'cherry', 'apple'])
print(np.sort(arr))
```

['apple' 'banana' 'cherry']

```
#Sort a boolean array:
import numpy as np
arr = np.array([True, False, True])
print(np.sort(arr))
```

[False True True]

- Sorting a 2-D Array

```
import numpy as np
arr = np.array([[3, 2, 4], [5, 0, 1]])
print(np.sort(arr))
```

[[2 3 4]
[0 1 5]]

NumPy Filter Array

1. Filtering Arrays

- Getting some elements out of an existing array and creating a new array out of them is called filtering.
- In NumPy, you filter an array using a boolean index list.
- If the value at an index is True that element is contained in the filtered array, if the value at that index is False that element is excluded from the filtered array.

```
#Create an array from the elements on index 0 and 2:
import numpy as np
arr = np.array([41, 42, 43, 44])
x = [True, False, True, False]
newarr = arr[x]
print(newarr)
```

```
[41 43]
```

- In the example above we hard-coded the True and False values, but the common use is to create a filter array based on conditions.

```
#Create a filter array that will return only values higher than 42:

import numpy as np

arr = np.array([41, 42, 43, 44])

# Create an empty list
filter_arr = []

# go through each element in arr
for element in arr:
    # if the element is higher than 42, set the value to True, otherwise False:
    if element > 42:
        filter_arr.append(True)
    else:
        filter_arr.append(False)

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)
```

```
[False, False, True, True]
[43 44]
```

- We can directly substitute the array instead of the iterable variable in our condition and it will work just as we expect it to.

```
#import numpy as np

arr = np.array([41, 42, 43, 44])

filter_arr = arr > 42

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)
```

```
[False False  True  True]
[43 44]
```